

Lösen der AsteroidStatic-Umgebung mit tiefem bestärkendem Lernen

Himanshu Mohan Nikam

Aufgabe 2 – Deep Reinforcement Learning
Prof. Thomas Nierhoff, OTH Amberg-Weiden
21. Juli 2026

1 Einleitung und Aufgabenstellung

Dieses Projekt trainiert Agenten des bestärkenden Lernens (Reinforcement Learning), um die bereitgestellte *AsteroidStatic*-Umgebung zu lösen. Der Agent ist ein Massepunkt (Raumschiff) in einer 2D-Arena mit 25 kreisförmigen Asteroiden. Er wird durch eine kontinuierliche Beschleunigung gesteuert und driftet aufgrund seiner Trägheit. Eine Episode ist erfolgreich, wenn das Raumschiff dem Ziel bis auf 0,30 nahekommt, und scheitert bei Kollision mit einem Asteroiden, beim Verlassen der Arena oder beim Erreichen des Zeitlimits von 500 Schritten.

Beobachtung: $\text{obs} = [x, y, v_x, v_y]$ (nur Position und Geschwindigkeit – Ziel- und Asteroidenpositionen sind *nicht* beobachtbar). **Aktion:** $a = [a_x, a_y]$, jeweils ein reeller Wert in $[-1, 1]$. **Belohnung** (bereits von der Umgebung geformt): $+0,5 \times$ Fortschritt zum Ziel pro Schritt, $+10$ für das Erreichen des Ziels und -100 für eine Kollision. Der Agent lernt ausschließlich aus Beobachtungen, Belohnungen und Info; er darf weder Hindernispositionen auslesen noch gegen die Geometrie planen. Alle bereitgestellten Dateien werden unverändert verwendet, und nur der Zufallskeim (Seed) wird variiert, um verschiedene Arenen zu erhalten.

Zwei bewertete Zielgrößen werden gemessen, jeweils gemittelt über zehn Arenen (Seeds 0–9): **(1) Stichprobeneffizienz** – die geringste Anzahl an Umgebungsaufrufen bis zum erstmaligen Erreichen des Ziels (Metrik 1); und **(2) Geschwindigkeit** – die geringste Anzahl an Zeitschritten bis zum Ziel nach dem Training (Metrik 2).

2 Ansätze

Da es sich um eine Navigationsaufgabe mit kontinuierlicher Regelung handelt, wurden zwei komplementäre Algorithmenfamilien untersucht und innerhalb jeder eine Progression implementiert.

2.1 Wertbasierte Methoden (DQN-Familie)

Zunächst wurde der DQN-Ansatz implementiert und erprobt. Deep-Q-Netze lernen einen Wert $Q(s, a)$ über eine endliche Aktionsmenge; daher wird die kontinuierliche Aktion in 9 Bang-Bang-Aktionen diskretisiert – alle Kombinationen von $\{-1, 0, +1\}$ für a_x und a_y . Vollschub-Aktionen sind zudem der schnellste Weg, eine Masse zu bewegen, was der Geschwindigkeitsmetrik zugutekommt. Ausgehend von DQN wurden die Standardverbesserungen ergänzt: *Double DQN* (DDQN), das Aktionsauswahl und -bewertung entkoppelt, um die Wertüberschätzung zu verringern, sowie *Dueling DDQN*, das das Netz in einen Zustandswert-Strom $V(s)$ und einen Vorteils-Strom $A(s, a)$ aufteilt. Alle drei nutzen Experience Replay, ein Zielnetz und ϵ -greedy-Exploration.

2.2 Methoden der kontinuierlichen Regelung

Die zweite Familie wirkt direkt auf die kontinuierliche Aktion und vermeidet die Diskretisierung. Ein deterministischer Actor $\mu(s)$ schlägt eine Aktion vor, ein Critic $Q(s, a)$ bewertet sie. Es wurde eine Progression aus drei Verfahren implementiert: ein Basis-Actor-Critic; *DDPG*, das dieses durch weiche (Polyak-) Zielaktualisierungen und zeitlich korreliertes Ornstein-Uhlenbeck-Rauschen stabilisiert; und *TD3*, das zwei Critics mit geclipptem Minimum-Ziel (gegen Überschätzung), Zielstrategie-Glättung (*target policy smoothing*) und verzögerte Actor-Updates ergänzt. Die kontinuierliche Regelung erlaubt eine feine, abgestufte Steuerung – einschließlich sanftem Bremsen – die die festen Vollschieb-Aktionen nicht ausdrücken können, und erzeugt daher tendenziell kürzere, glattere Wege.

2.3 Betrachtete Methoden

Ebenfalls betrachtet, aber nicht im Fokus: *Deep SARSA*, ein on-policy-Verfahren, das in der Nähe der hohen -100 -Kollisionsstrafe von Natur aus risikoscheuer ist; sowie *Soft Actor-Critic* (SAC) und *PPO* als alternative Verfahren der kontinuierlichen Regelung mit stärkerer, entropiegetriebener Exploration.

3 Wichtige Implementierungstechniken

Mehrere praktische Maßnahmen waren entscheidend, damit die Verfahren in dieser hindernisdichten Umgebung mit seltener Zielbelohnung überhaupt lernen:

- **Eingangsnormalisierung.** Positionen und Geschwindigkeiten werden anhand der bekannten Umgebungsgrenzen auf einen vergleichbaren Bereich skaliert. Dies ist eine reine Eingangsskalierung (keine Analyse der Umgebung) und trug am stärksten zu stabilem Training bei.
- **Belohnungsbehandlung.** Die -100 -Kollisionsstrafe dominiert den kleinen $+0,5$ -Fortschrittsterm um rund zwei Größenordnungen, was die gelernten Wertschätzungen kollabieren lässt. Ein asymmetrisches Belohnungs-Clipping begrenzt die Kollisionsstrafe nach unten, hält aber den $+10$ -Zielbonus groß und aussagekräftig, sodass die seltenen erfolgreichen Übergänge das Lernen stark beeinflussen. Diese bewusste Umformung wird als Entwurfsentscheidung vermerkt.
- **Exploration.** Die Aufgabe ist explorationsschwer: Ein rein zufälliger Agent erreicht das Ziel nur in etwa $0,2\%$ der Episoden und auf mehreren Arenen nie, da das Ziel hinter dichten Hindernissen mindestens drei Einheiten entfernt liegt. Daher wird die Exploration stark und dauerhaft gehalten – Injektion von Zufallsaktionen mit voller Magnitude für die wertbasierten Agenten und korreliertes Ornstein-Uhlenbeck-Rauschen für die kontinuierlichen Agenten –, damit der Agent das Ziel weiter entdeckt, statt in eine triviale “Nichtstun”- oder “Flucht in den freien Raum”-Strategie zu kollabieren.
- **Stabilität.** Gradienten-Clipping, Zielnetze und eine Replay-Aufwärmphase wurden durchgängig eingesetzt.

4 Experimenteller Aufbau

Alle Agenten sind in PyTorch implementiert und nutzen die Standardeinstellungen der Umgebung; nur der Seed wird geändert. Die Hyperparameter werden mit *Optuna* optimiert. Gemäß der Aufgabenregel erfolgt das Tuning auf Seeds, die von den Evaluations-Seeds disjunkt sind: Aus den Kandidaten 100–119 wurden die beiden am besten lösbaren (Seeds 115 und 119, auf denen das

Ziel zumindest gelegentlich erreichbar ist) für die Suche verwendet; die Endergebnisse werden auf den Seeds 0–9 berichtet. Die Zielfunktion minimiert die Aufrufe bis zur ersten Lösung, mit einer abstands-basierten Rückfalloption, sodass Konfigurationen, die im Episodenbudget nicht lösen, dennoch nach ihrer Annäherung an das Ziel bewertet werden. Je Lauf werden Metrik 1, die kürzeste erfolgreiche Episodenlänge (Metrik 2) sowie die zu dieser kürzesten Episode gehörenden Aufrufe erfasst.

Tabelle 1 fasst die besten von Optuna gefundenen Konfigurationen zusammen.

Tabelle 1: Getunte Hyperparameter (beste Optuna-Konfiguration je Verfahren).

Hyperparameter	Actor-Critic	DDPG	TD3
actor_lr	$4,3 \cdot 10^{-5}$	$1,7 \cdot 10^{-5}$	$1,4 \cdot 10^{-4}$
critic_lr	$1,3 \cdot 10^{-4}$	$4,8 \cdot 10^{-4}$	$1,9 \cdot 10^{-3}$
γ	0,979	0,992	0,962
Batch-Größe	64	128	128
Hidden-Größe	256	256	256
τ (Polyak)	–	$1,8 \cdot 10^{-3}$	$8,0 \cdot 10^{-3}$
Explorationsrauschen	$\sigma_{\text{Gauss}} = 0,17$	$\sigma_{\text{OU}} = 0,37$	$\sigma_{\text{OU}} = 0,32$
ε_{min} / -zerfall	0,22 / 0,995	–	–
target_update_freq	899	–	–
policy_noise / noise_clip	–	–	0,34 / 0,50
policy_delay	–	–	1

5 Ergebnisse

Jedes Verfahren wurde mit seiner besten Konfiguration auf den Seeds 0–9 evaluiert (bis zu 5000 Episoden je Seed). Tabelle 2 zeigt die Robustheit (gelöste Seeds) und die über die *jeweils gelösten* Seeds gemittelten Metriken.

Tabelle 2: Hauptergebnisse, gemittelt über die je Verfahren gelösten Seeds.

Verfahren	Gelöste Seeds	Ø Metrik 1 (Aufrufe bis Lösung)	Ø Metrik 2 (Schritte)	Ø zugeh. Aufrufe
Actor-Critic	6/10	80.847	26,0	130.101
DDPG	5/10	4.473	24,2	88.761
TD3	10/10	89.295	37,9	147.919

Da jedes Verfahren nur über die von ihm gelösten Seeds gemittelt wird, sind diese Mittelwerte *nicht direkt vergleichbar*: TD3 löst als einziges auch die beiden schwierigsten Seeds (4 und 8) mit langen Wegen, was seine Mittelwerte erhöht; DDPGs niedrige Metrik 1 beruht teils darauf, dass es nur fünf, eher leichte Seeds löst. Tabelle 3 vergleicht die Verfahren daher fair auf der gemeinsamen Teilmenge der von allen dreien gelösten Seeds (2, 5, 7, 9).

Tabelle 3: Fairer Vergleich auf den von allen Verfahren gelösten Seeds (2, 5, 7, 9).

Verfahren	Ø Metrik 1	Ø Metrik 2	Ø zugeh. Aufrufe
Actor-Critic	24.304	24,8	51.619
DDPG	4.449	24,8	105.190
TD3	4.667	24,0	35.535

Tabelle 4 listet die Einzelergebnisse je Seed (“–” = innerhalb des Budgets nicht gelöst).

Tabelle 4: Ergebnisse je Seed: Metrik 1 (Aufrufe bis Lösung) / Metrik 2 (Schritte).

Seed	Actor-Critic		DDPG		TD3	
	Metrik 1	M2	Metrik 1	M2	Metrik 1	M2
0	–	–	–	–	121.118	26
1	13.159	25	–	–	32.362	23
2	75.105	26	1.235	27	10.022	25
3	–	–	–	–	250.901	35
4	–	–	–	–	144.963	83
5	10.166	27	3.185	26	3.192	25
6	374.705	32	–	–	45.808	24
7	6.738	23	7.228	23	3.151	23
8	–	–	4.573	22	279.128	92
9	5.207	23	6.146	23	2.301	23

6 Diskussion

Die entscheidende Schwierigkeit ist die Exploration, nicht die Lernregel. Da das Ziel hinter einem dichten Hindernisfeld liegt, kann der geradlinige Fortschrittsterm den Agenten aktiv in die Asteroiden direkt vor dem Ziel locken; Zustände näher am Ziel tragen dann ein höheres Kollisionsrisiko und einen niedrigeren Wert, und ein Agent kann rational lernen, in den freien Raum zu fliehen oder – bei der deterministischen kontinuierlichen Strategie – eine nahezu null Aktion (“Nichtstun”) auszugeben und bis zum Zeitlimit zu überleben. Dies zu überwinden erforderte eine Exploration, die stark genug ist, das Ziel oft genug tatsächlich zu erreichen.

Über alle Seeds war **TD3 am robustesten** und löste als einziges alle zehn Arenen. Auf der fair vergleichbaren gemeinsamen Teilmenge (Tabelle 3) sind **DDPG und TD3 deutlich stichprobeneffizienter als Actor-Critic** (≈ 4.500 gegenüber ≈ 24.300 Aufrufen bis zur Lösung), und TD3 liefert zugleich die kürzesten Wege (Metrik 2 = 24,0) und die wenigsten zugehörigen Aufrufe. Die erhöhten Gesamtmittelwerte von TD3 in Tabelle 2 stammen ausschließlich von den schweren Seeds 4 und 8, die lange Wege (83 bzw. 92 Schritte) erfordern – der Preis dafür, dass diese überhaupt gelöst werden.

Zwei Dinge, die *nicht* funktionierten, sind berichtenswert. Symmetrisches Belohnungs-Clipping auf $[-1, 1]$ ließ die Kollisionsstrafe groß genug, um alle Wertschätzungen kollabieren zu lassen. Und schwaches Gauß-Rauschen um eine bereits kollabierte deterministische Strategie zittert nur lokal und reicht nie weit genug, um das Ziel zu finden; starke, korrelierte oder vollmagnitudige Exploration war notwendig. Der erprobte DQN-Ansatz konnte einzelne Arenen zuverlässig lösen und profitierte von der vollmagnitudigen ε -greedy-Exploration über die diskreten Vollschieb-Aktionen.

7 Fazit

Es wurden zwei Familien tiefer RL-Agenten auf der AsteroidStatic-Aufgabe implementiert und verglichen – eine wertbasierte DQN/DDQN/Dueling-DDQN-Progression und eine kontinuierliche Actor-Critic/DDPG/TD3-Progression – zusammen mit den Explorations- und Belohnungsformungstechniken, die zum Lernen in dieser explorationsschweren Umgebung nötig waren. **TD3 erwies sich als das robusteste Verfahren** (10/10 gelöste Arenen) und war auf vergleichbaren Seeds sowohl stichproben- als auch wegeffizient; DDPG war auf den leichteren Arenen ebenfalls sehr stichprobenineffizient, aber weniger robust, und der Basis-Actor-Critic war das schwächste der drei kontinuierlichen Verfahren.

Literatur

- [1] V. Mnih et al., *Human-level control through deep reinforcement learning*, Nature 518, 2015.
- [2] H. van Hasselt, A. Guez, D. Silver, *Deep Reinforcement Learning with Double Q-learning*, AAAI 2016.
- [3] Z. Wang et al., *Dueling Network Architectures for Deep Reinforcement Learning*, ICML 2016.
- [4] T. P. Lillicrap et al., *Continuous control with deep reinforcement learning* (DDPG), ICLR 2016.
- [5] S. Fujimoto, H. van Hoof, D. Meger, *Addressing Function Approximation Error in Actor-Critic Methods* (TD3), ICML 2018.